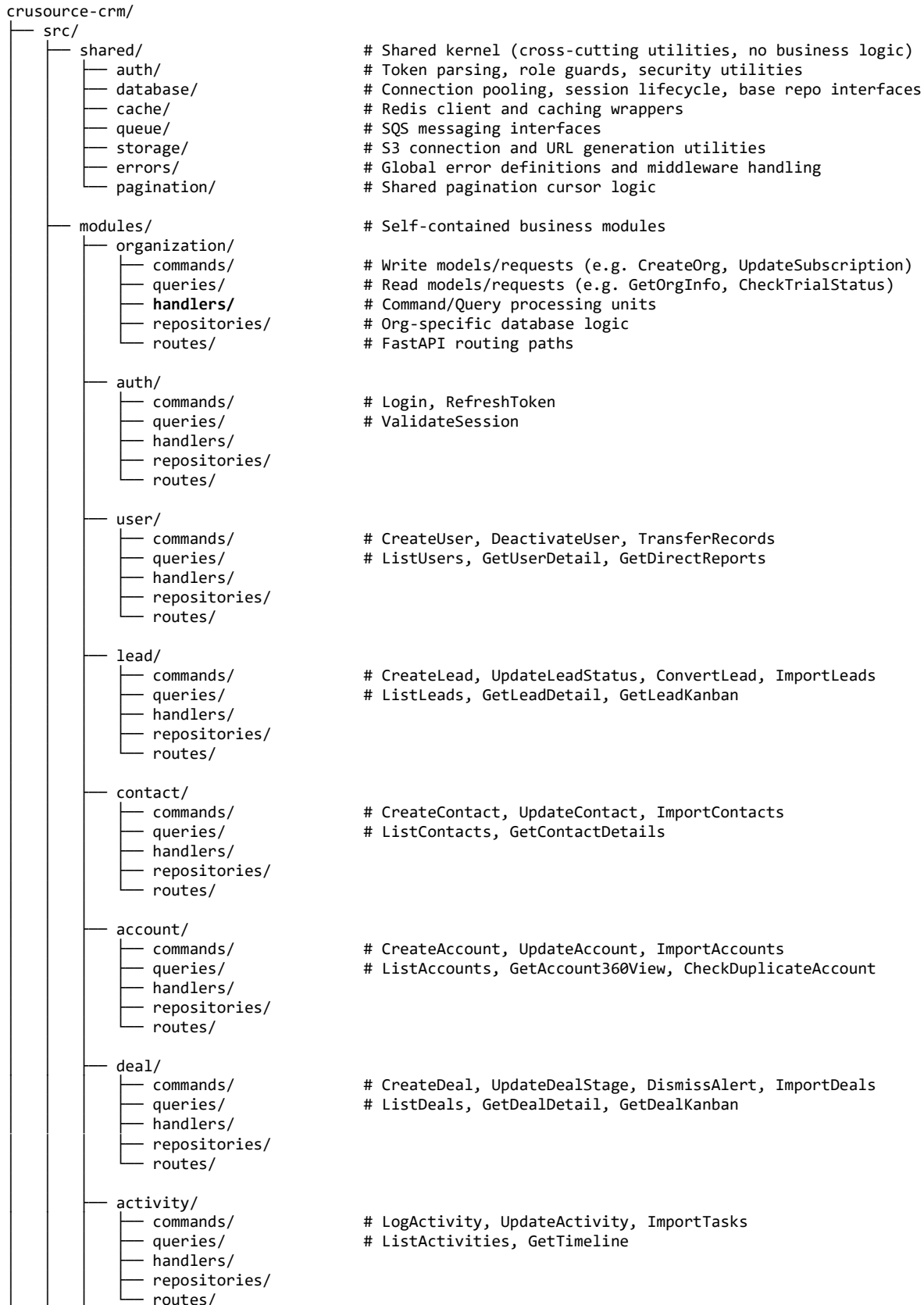


Crusource CRM

LLD

1. Project Folder Structure

Here is the folder structure for the project backend. It follows the CQRS organization pattern where commands, queries, and handlers are split at the folder level:



```

├── document/
│   ├── commands/           # RequestUploadUrl, CompleteUpload, DeleteDocument
│   ├── queries/            # ListDocuments, RequestDownloadUrl
│   ├── handlers/
│   ├── repositories/
│   └── routes/
├── dashboard/
│   ├── queries/            # GetHomeDashboard, GetAnalytics, RunReport
│   ├── handlers/
│   ├── repositories/
│   └── routes/
├── ai/
│   ├── commands/           # TriggerEmailDraft, ChatRequest
│   ├── queries/            # GetEmailDraft, GetChatHistory
│   ├── handlers/
│   ├── repositories/
│   └── routes/
├── notification/
│   ├── commands/           # CreateNotification, MarkNotificationAsRead
│   ├── queries/            # GetUserNotifications
│   ├── handlers/
│   ├── repositories/
│   └── routes/
├── search/
│   ├── queries/            # ExecuteGlobalSearch
│   ├── handlers/
│   ├── repositories/
│   └── routes/
├── settings/
│   ├── commands/           # SavePipelineStages, SaveScoringWeights
│   ├── queries/            # GetPipelineConfig, GetScoringConfig
│   ├── handlers/
│   ├── repositories/
│   └── routes/
└── main.py                 # Root application entry
```

2. Technology Stack

Frontend

Layer	Technology	Purpose
Framework	Next.js 14+ (App Router) + TypeScript	SSR, file-based routing
UI	Tailwind CSS + shadcn/ui	Styling + accessible components
Server State	TanStack Query 5	Caching, refetching, optimistic updates
Client State	Redux Toolkit 2	Complex cross-component state
Tables	TanStack Table 8	Headless sortable/filterable tables
Drag & Drop	dnd-kit 6	Kanban board interactions
Forms	React Hook Form 7 + Zod 3	Form management + schema validation
Charts	Recharts 2	Dashboard visualizations

Backend

Layer	Technology	Purpose
Framework	FastAPI 0.110+	Async HTTP, auto-generated OpenAPI
Language	Python 3.12+	AI/ML ecosystem
ORM	SQLAlchemy 2 (async)	Query building, model mapping
Migrations	Alembic 1.13+	Schema versioning
Auth	python-jose (HS256 JWT) + Passlib (Argon2id)	Token auth + password hashing
Validation	Pydantic 2	Request/response validation
HTTP Client	httpx 0.27+	Async calls to Claude API

Infrastructure

Layer	Technology	Purpose
Frontend Hosting	Vercel	Zero-config Next.js deployment
Backend Hosting	AWS ECS / Fargate	Serverless containers
Database	Amazon RDS PostgreSQL 16 + pgvector	Relational storage + vector search
Cache	AWS ElastiCache (Redis 7.x)	Dashboard cache, rate limiting
Object Storage	AWS S3	Document binaries
Message Queue	AWS SQS (Standard)	Async job dispatch
Secrets	AWS Secrets Manager	Credentials, JWT secret
CI/CD	GitHub Actions	Build → test → deploy
Monitoring	AWS CloudWatch	Logs, metrics, alarms

3. API Design

Conventions

Aspect	Convention
Base URL	https://api.crusource.com/v1
Auth	Authorization: Bearer <JWT>
Pagination	Cursor-based: ?cursor=<uuid>&limit=50
Filtering	Query params: ?status=new&source=referral
Sorting	?sort_by=created_at&sort_order=desc
Error format	{ "error": { "code", "message", "details", "request_id" } }

Endpoint Catalog

Auth & Users (Admin-only for user mgmt)

Method	Path	Description
POST	/auth/login	Login → JWT
POST	/auth/refresh	Refresh JWT
GET/POST/PATCH	/users, /users/{id}	User CRUD (Admin)
POST	/users/{id}/deactivate	Deactivate (preserves records)
POST	/users/{id}/transfer	Transfer all records

Leads

Method	Path	Description
GET/POST	/leads	List (scoped) / Create
GET/PATCH/DELETE	/leads/{id}	Read / Update / Delete
POST	/leads/import	CSV bulk import
POST	/leads/{id}/convert	Convert → Account + Contact + Deal (atomic)
GET	/leads/{id}/score	AI lead score
GET	/leads/kanban	Grouped by status

Contacts

Method	Path	Description
GET/POST	/contacts	List / Create
GET/PATCH/DELETE	/contacts/{id}	CRUD
POST	/contacts/import	CSV import

Accounts

Method	Path	Description
GET/POST	/accounts	List / Create (triggers duplicate check)
GET/PATCH/DELETE	/accounts/{id}	CRUD
POST	/accounts/import	CSV import
GET	/accounts/{id}/contacts	Contacts tab
GET	/accounts/{id}/deals	Deals tab
GET	/accounts/{id}/activities	Activities tab
GET	/accounts/duplicates	Duplicate name check

Deals

Method	Path	Description
GET/POST	/deals	List / Create
GET/PATCH/DELETE	/deals/{id}	CRUD (PATCH includes stage change)
POST	/deals/import	CSV import
GET	/deals/kanban	Grouped by stage with column totals
GET	/deals/{id}/risk-alerts	Risk alerts for deal
POST	/deals/{id}/risk-alerts/{alert_id}/dismiss	Dismiss alert

Activities

Method	Path	Description
GET/POST	/activities	List (filterable by type) / Create
GET/PATCH/DELETE	/activities/{id}	CRUD
POST	/activities/import	CSV import (tasks)
GET	/activities/timeline/{entity_type}/{entity_id}	Unified timeline

Documents

Method	Path	Description
GET	/documents	List (scoped)
POST	/documents/upload-url	Get pre-signed upload URL
POST	/documents	Register metadata after upload
GET	/documents/{id}/download-url	Get pre-signed download URL
DELETE	/documents/{id}	Delete (metadata + S3 object)

Dashboards & Reports

Method	Path	Description
GET	/dashboard/home	Home dashboard (role-scoped)
GET	/dashboard/analytics	Org-wide analytics
GET	/reports	List available reports
GET	/reports/{slug}	Execute pre-built report

AI

Method	Path	Description
POST	/ai/email-draft	Request email draft (async → SQS) → 202
GET	/ai/email-draft/{id}	Get generated draft
POST	/ai/chat	Send message to RAG chatbot
GET	/ai/chat/sessions/{id}	Get chat history

Settings (Admin), Search & Notifications

Method	Path	Description
GET/PUT	/settings/pipeline-stages	Read / bulk-replace stages
GET/PUT	/settings/scoring-weights	Read / update scoring weights
GET	/search?q=<query>	Global search (leads + contacts + accounts + deals)
GET	/notifications	List user notifications
PATCH	/notifications/{id}/read	Mark as read

4. Authentication & Authorization

JWT Structure

Field	Value
sub	User UUID
org_id	Organization UUID (tenant isolation)
role	admin / sales_manager / sales_rep
exp	Expiry (24h default)
Signing	HS256, secret from AWS Secrets Manager

Auth Flow

Client logs in with email and password. The API validates the user against the database (scoped by organization and active status), verifies the password with Argon2, and issues a signed JWT containing the subject, org_id, role, and expiry. All subsequent requests must include the token as "Authorization: Bearer <JWT>".

Row-Level Scope Filter

Applied at the ORM layer on every data query. Each request's JWT determines tenant scope (org_id) first, then role-based visibility:

Role	Visibility
admin	No further filter — sees all organization data
sales_manager	WHERE owner_id IN (self + direct_reports)
sales_rep	WHERE owner_id = self

Endpoint Permission Guards

Endpoint Group	Allowed Roles
User management, settings	admin only
All data CRUD, AI features	admin, sales_manager, sales_rep (data scoped by role)
Login	Unauthenticated

5. Module Data Flows

5.1 Lead Conversion (Atomic Transaction)

A lead conversion is executed as a single atomic database transaction. The API first checks for a duplicate account by name (case-insensitive match). If a duplicate is found, the request is rejected with a 409 Conflict and the existing account ID. Otherwise, the API inserts a new account, then a contact linked to that account, then a deal linked to both, and finally updates the originating lead's status to "converted" along with references to the newly created records. All of these steps are committed together, or none are.

5.2 Deal Pipeline

Deals move through a fixed sequence of stages: Qualification → Discovery → Proposal → Negotiation, with Negotiation resolving to either Closed Won or Closed Lost.

Note: *Stage update side effects: update last_interaction_at, invalidate dashboard cache, clear stale-deal risk alerts.*

5.3 Activity Timeline

A single query on the activities table (single-table inheritance) filtered by related_to_type and related_to_id, ordered by created_at descending, returns tasks, meetings, calls, and notes together in one unified response.

5.4 Document Upload/Download

Uploads and downloads both use pre-signed S3 URLs so binaries never pass through the API server:

Upload: the client requests an upload URL (providing file name and content type), receives a pre-signed PUT URL valid for 15 minutes, uploads the file directly to S3, then registers the resulting S3 key and metadata with the API, which returns the new document ID.

Download: the client requests a download URL for a document ID and receives a pre-signed GET URL valid for 15 minutes, then fetches the file directly from S3.

Note: *Binaries never pass through the API server. S3 key format: {org_id}/{related_type}/{related_id}/{uuid}/{filename}*

6. AI Subsystem

Architecture Overview

The AI subsystem splits into two categories: synchronous, rule-based logic with no LLM involvement (lead scoring, deal risk alerts), and asynchronous, LLM-backed features dispatched through SQS (email drafting, RAG chatbot). Rule-based features run on cron schedules and event triggers directly against PostgreSQL. LLM-backed features enqueue jobs to the ai-jobs SQS queue, which are picked up by workers that build a prompt with the relevant record context, call the Claude API, and persist results back to PostgreSQL. The RAG chatbot additionally performs an approximate-nearest-neighbor search over pgvector to retrieve the top-k relevant document chunks before querying Claude.

Feature Breakdown

Feature	Type	Trigger	Processing
Lead Scoring	Rule-based (no LLM)	Overnight cron + instant on status/activity change	Weighted signal evaluation → 0–100 score + plain-language reason. Admin-tunable weights via scoring_weights table.
Deal Risk Alerts	Rule-based (no LLM)	Daily cron scan	Flags deals with no activity ≥7 days or close date ≤7 days without recent progress. Stored in deal_risk_alerts, dismissible.
Email Drafts	Generative (Claude)	User clicks "Draft with AI"	API enqueues to SQS → Worker fetches record + activity context → builds prompt → Claude → result stored in ai_email_drafts. Returns 202; client polls for result.
RAG Chatbot	Generative (Claude + pgvector)	User sends message	Embed query → ANN search in document_embeddings (scoped by org_id + user visibility) → top-k context chunks → Claude prompt → answer with source_record_ids.

Embedding Pipeline

Trigger: a record create or update event enqueues an embedding job to the embeddings SQS queue.

Worker: fetches the record text, chunks it into 512-token segments, generates embeddings, and upserts them into document_embeddings.

Scope: RAG queries filter embeddings by org_id and the user's visible record IDs — the same scope filter used for direct queries.

Guardrails

- AI never acts autonomously — it produces scores, drafts, and flags only.
- AI output inherits the same visibility scope as the underlying record.
- Every chatbot answer includes source record links.

7. Caching Strategy

Layer: ElastiCache (Redis 7.x). Pattern: cache-aside with write-through invalidation.

Cache Target	Key Pattern	TTL	Invalidation
Home Dashboard	dash:home:{org_id}:{user_id}	5 min	Activity/deal/lead write
Analytics Dashboard	dash:analytics:{org_id}	10 min	Deal/lead write
Kanban (Leads/Deals)	kanban:{type}:{org_id}:{user_id}	2 min	Status/stage change
Pre-built Reports	report:{slug}:{org_id}:{user_id}	15 min	Underlying data change
User's direct reports	reports:{user_id}	30 min	Admin "Reports To" change
Rate limit counters	ratelimit:{user_id}:{endpoint}	1 min sliding	—

Invalidation: on any write operation, delete relevant cache keys for the affected org_id + user_id.

8. File Storage (S3)

Setting	Value
Key format	{org_id}/{related_type}/{related_id}/{uuid}/{filename}
Pre-signed URL expiry	15 minutes (both upload and download)
Max file size	50 MB (enforced via S3 policy)
Encryption	SSE-S3 (AES-256) at rest
Versioning	Disabled (V1)
CORS	Allow PUT/GET from frontend domain only
Public access	Blocked — all access via pre-signed URLs

9. Background Job Processing

Queue Topology

Queue	Purpose	Workers	DLQ
ai-jobs	Email drafts, batch scoring, risk scans	ECS Fargate tasks	ai-jobs-dlq
notifications	Notification generation & delivery	ECS Fargate tasks	notifications-dlq
embeddings	Vector embedding generation	ECS Fargate tasks	embeddings-dlq

Job Message Schema

`{ job_id, job_type, org_id, user_id, payload: { record_type, record_id, context }, enqueued_at }`

Worker Config

Setting	Value	Rationale
Long-poll wait	20s	Reduce empty receives
Visibility timeout	300s (5 min)	Buffer for LLM calls (10–30s)
Max receive count	3	3 attempts before DLQ
DLQ retention	14 days	Time for manual inspection/replay

Scaling

Workers auto-scale via ECS based on `ApproximateNumberOfMessagesVisible > 10`.

10. Search & Notifications

Global Search

- UNION ALL query across leads, contacts, accounts, deals using ILIKE matching on name/email/company fields.
- Scoped by org_id + user visibility, limited to top 20 results.
- Upgrade path: migrate to tsvector + GIN indexes if performance degrades.

Notifications

Event	Notification Type	Recipient
Task assigned	assignment	Assigned user
Task due date reached	task_due	Task owner
Deal risk alert created	deal_risk	Deal owner

Delivery: client polls GET /notifications every 30s. WebSocket push is a post-V1 upgrade.

11. Security

Threat	Mitigation
Credential theft	Argon2id hashing, short-lived JWT, secrets in Secrets Manager
SQL injection	SQLAlchemy parameterized queries — no raw string SQL
XSS	React auto-escaping + Content Security Policy headers
Broken access control	Row-level scope filter on every query + endpoint role guards
Tenant leakage	org_id filter at ORM layer on ALL queries
File upload attacks	Pre-signed URLs with content-type enforcement, 50 MB limit
Rate limiting	ElastiCache sliding-window per user+endpoint
Insecure deserialization	Pydantic strict validation on all request bodies
Client-side code execution	No eval(), hardened CSP

Rate limits: Login: 10/min. AI endpoints: 20–30/5min. Default: 100/min.

Security headers: HSTS, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, strict CSP, Referrer-Policy.

12. Scalability & Performance

Scaling Strategy

Component	Method	Trigger
API Server	ECS horizontal (task count)	CPU > 70% for 3 min
Workers	ECS horizontal (task count)	Queue depth > 10
DB reads	Read replica routing	Report/dashboard queries
DB writes	Vertical (instance size)	Write throughput bottleneck
Cache	ElastiCache cluster sharding	Memory pressure

Performance Targets

Metric	Target
API p50 / p95 / p99	< 100ms / < 500ms / < 1s
Dashboard load (cache hit / miss)	< 2s / < 5s
Global search	< 500ms
AI email draft	< 30s (async)
AI chat response	< 10s
CSV import (1000 rows)	< 30s

Key Optimizations

- Cursor-based pagination (not OFFSET).
- SQLAlchemy selectinload / joinedload for N+1 prevention.
- Kanban: single GROUP BY stage_id + SUM(amount) query.
- Dashboard: cache-first with TTL invalidation.
- Connection pooling: 30 connections/instance, PgBouncer if scaling beyond ~6 instances.

13. Observability

Logging

- Backend: structlog → CloudWatch Logs (JSON structured).
- Frontend: Vercel Log Drain.
- Every request logs: request_id, user_id, org_id, method, path, status, duration_ms, scope.
- Never logged: passwords, tokens, PII in query params.

Alarms

Metric	Source	Threshold
5xx error rate	ALB	> 1% for 5 min
API latency (p95)	ALB	> 1s for 5 min
ECS CPU	ECS	> 80% for 5 min
RDS connections	RDS	> 150 / 200
SQS DLQ count	SQS	> 0 (immediate)
Cache hit rate	ElastiCache	< 80%

Health Check

GET /health checks DB, cache, and SQS connectivity and returns { status: "healthy" | "degraded", checks: {...} }.

14. Error Handling

Response Format

```
{ "error": { "code", "message", "details": [...], "request_id" } }
```

Error Codes

Code	HTTP	When
VALIDATION_ERROR	422	Pydantic validation failure
AUTHENTICATION_REQUIRED	401	Missing / expired JWT
FORBIDDEN	403	Insufficient role
NOT_FOUND	404	Record missing or out of scope
DUPLICATE_RESOURCE	409	Duplicate account detected
RATE_LIMITED	429	Rate limit exceeded
AI_GENERATION_FAILED	502	Claude API failure
INTERNAL_ERROR	500	Unhandled exception
SERVICE_UNAVAILABLE	503	DB / cache / SQS down

15. Appendix — ADRs

#	Decision	Chosen	Alternative	Rationale
001	Frontend	Next.js (App Router)	React (Vite)	SSR, file routing, Vercel deploy
002	Backend	Python (FastAPI)	Node.js (NestJS)	AI/ML ecosystem maturity
003	Database	PostgreSQL + pgvector	MongoDB	Relational integrity + vector search in same DB
004	Activity storage	Single-table inheritance	Separate tables	Simplifies timeline query (no UNION ALL)
005	Activity linking	Polymorphic (type + id)	Multiple nullable FKs	Flexible, avoids N nullable columns
006	File storage	Pre-signed URL (S3 direct)	API proxy	Decouples file size from API memory
007	AI processing	Async (SQS workers)	Synchronous	LLM calls (10–30s) can't block requests
008	Cache	ElastiCache (managed)	Self-managed Redis	Reduced ops burden
009	Job queue	SQS	Celery + Redis	Serverless, no broker dependency
010	Lead scoring	Rule-based	LLM-based	Deterministic, fast, Admin-tunable, no AI cost
011	Deal risk alerts	Rule-based	LLM-based	Simple thresholds sufficient
012	Vector store	pgvector	Pinecone / Weaviate	No extra infra; fine for V1 scale
013	Auth	JWT (stateless)	Sessions	Horizontal scalability
014	Search	ILIKE	Elasticsearch	Sufficient for V1; avoids extra infra
015	Notifications	HTTP polling	WebSockets	Simpler for V1; WebSockets post-V1